DEEPDIVES

AI's impact on software engineers in 2026: key trends, Part 2

Tradeoffs of AI tooling, why adopting AI at company-level is hard, what's changed in two years, and more. The third and final part of a series analyzing our 2026 AI survey results



GERGELY OROSZ AND ELIN NILSSON

MAY 19, 2026 · PAID

204 3 16 Share

Earlier this year, we asked The Pragmatic Engineer subscribers about the AI tools you use, how you use them, and, crucially, what you think of them. We received more than 900 responses, and with this article we conclude our analysis of that data, covering:

1. **Tradeoffs of AI tooling.** Positive and negative sentiments, like how AI tools often mean less time spent on tedious, repetitive work, but that it often leads to unrealistic business expectations.
2. **Adopting AI at scale is hard.** Companies are struggling to achieve adoption that feels productive at the team and org levels. Our survey finds the benefits of AI heavily depend on the engineering culture that was in place before.

3. **Impact on codebase quality.** Codebase quality seems to be decreasing, but management at most places does not care. Meanwhile, maintenance duty is falling upon a shrinking number of engineers who still understand increasingly complex codebases.
4. **Less experienced engineers and AI.** For these folks, AI seems to be less helpful and they rack up higher AI token bills. Maybe they need more space to learn, mentorship, and support.
5. **AI tooling 'addiction'.** Using AI agents "feels like a slot machine" encouraging "just one more prompt"-type behavior, while some folks think that the pricing of plans is built in a way to "lure" them to prompt more and more.
6. **Changes since 2024.** Fewer devs are negative about AI, but there's not all that much more positivity. The models have become much higher quality, and better tooling improves trust.
7. **Where are we headed?** The concept of code ownership seems to be eroding, and collaboration within teams becomes less important.

Previously in this series, we covered:

- **AI tooling for software engineers in 2026**: a detailed summary of survey responses, covering the most-used AI tools, trends, AI agent usage, company size and usage, and tools engineers love.

- **The impact of AI on software engineers in 2026: key trends. Part 1**. Concerns about mounting AI costs, more engineers hitting usage limits, and AI tools having uneven effects upon different types of engineers

Full subscribers can access a more detailed report.

1. AI tooling tradeoffs

What can be said about the impact of AI tooling on engineers this year? Well, based on the responses that readers sent in to our survey, there are some common patterns:

Code and output tradeoffs

Area	✅ Positive impacts	❌ Negative impacts
Increased speed and output: more code, pull requests, docs, etc	• **Less toil**: spending less time on tedious, repetitive, undesirable work • **More time for important stuff:** Extra time for higher-level and enjoyable tasks • **Tighter feedback loops:** More "flow state" time, too • **More scope to do neglected tasks:** smaller things less overlooked due to lower priority or impact	• **Overwhelming output volume:** Rapidly growing codebases, a flood of PRs and code reviews • **CI/CD load** higher than ever • **Inflated business expectations** due to the higher output • **Being overwhelmed** by reviewing a lot and doing more in general
Less tooling friction: Lower friction with developer tooling, simpler onboarding to new or unknown codebases and environments	• **Faster onboarding** to new codebases, projects and environments • **More hands on deck:** more devs can contribute to more stuff • **Hands-on managers:** managers and leaders can engage more hands-on with code	• **More "orphaned" parts of codebase:** increase in code nobody fully understands or can own in the traditional sense • **Codebase decay:** subtle bugs, tech debt, and bad patterns accumulate • **Lower quality:** decrease in idiomatic code and language-specific best practices
"Code is cheap": Doing and unlocking tasks there wouldn't be worth it without AI	• **More tools built:** more custom scripts, small custom programs, quality-of-life improvements & one-off solutions • **More building outside of work:** building stuff for and with family, friends, kids, local community etc. • **Fixing stuff that wasn't important before:** fixing bugs, adding & tweaking features that didn't used to be worth doing	• **Small stuff isn't production-ready:** "low stake software" could prove risky if it spreads, as it likely won't be up to production standards, e.g., security • **Tech debt accumulates in production codebases:** a small task can balloon into a big one: e.g. bug fixes that require substantial re-writes or feature tweaks may cause inconsistencies

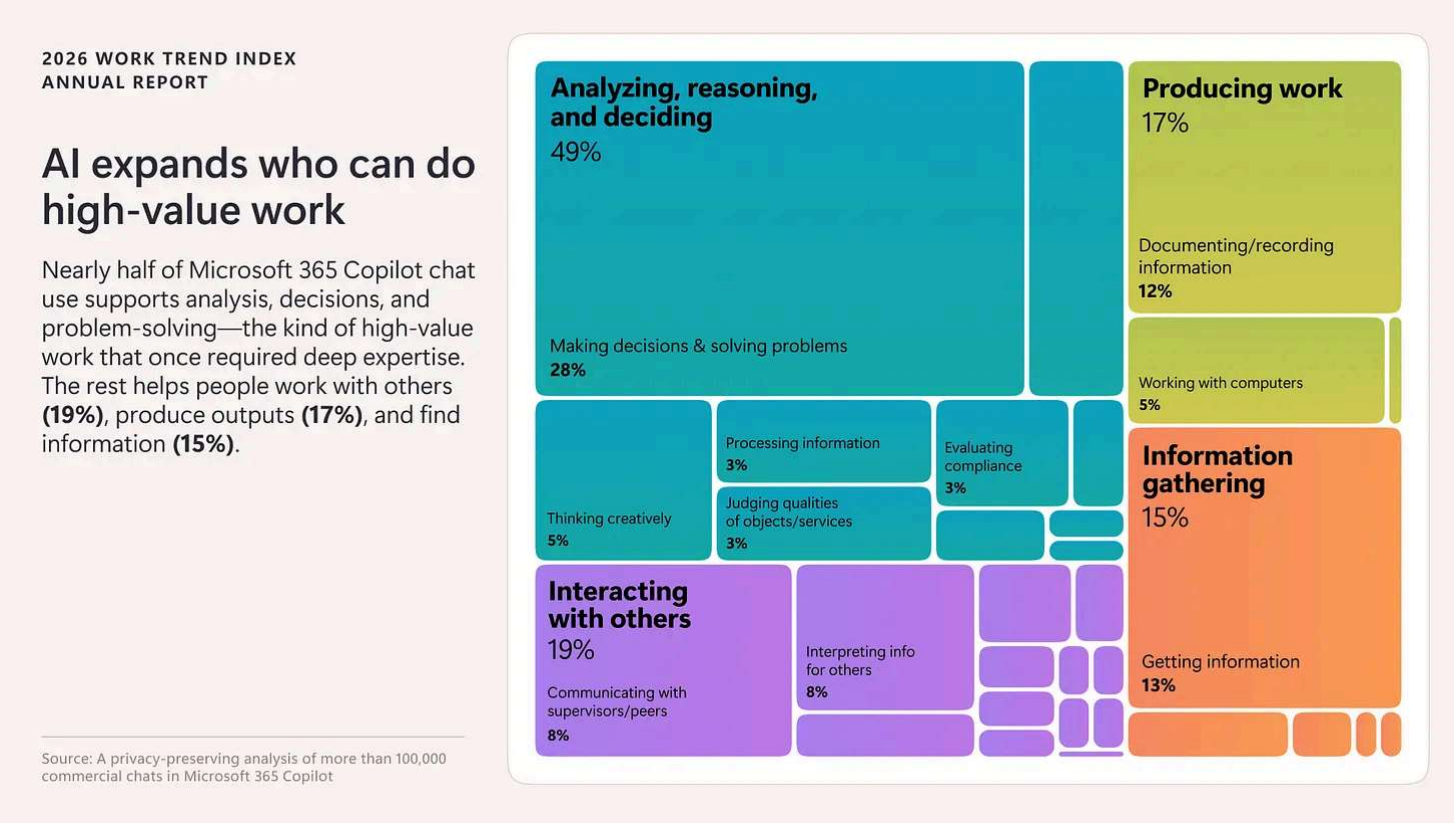


Productivity tradeoffs

Area	✅ Positive impacts	❌ Negative impacts
Easier to get unblocked Access to help, answers, information & knowledge without interacting with other devs	• **More deep work and focus** thanks to fewer interruptions and context switches • **No "stupid" questions:** reduces friction • **Unblocking yourself:** even without peer knowledge, it's harder to get truly stuck	• **Less shared learning**: LLMs dampen opportunity for folks to learn stuff with each other • **Fewer constructive discussions and disagreements?** LLMs are architecturally sycophantic and agreeable
Cheaper & faster to build: prototyping, creating, testing software	• **Building**: devs can build and prototype faster • **Cheaper & faster**: lowers costs of testing and communicating ideas effectively • **Cross-platform collaboration**: potential to deepen understanding and collaborate across disciplines • **Non-devs fix bugs**: allows PMs/designers/non-devs to get hands-on	• **Building the wrong thing**: easy for bad ideas and poor features to reach fruition that shouldn't be built • **AI slop**: code changes and code reviews with low / questionable quality • **More bugs**: seemingly simple bug fixes can have complex side effects. Non-developers might miss these • **Less communication**: expertise and domain knowledge overlooked in favor of asking AI
The "10x" effect: some individuals "10x" their output with parallel agents, multiple concurrent workstreams, AI as a "second brain", etc	• **Decreased cost of context switching** because AI can help with heavy lifting • **More output without more input:** no proportional increase in time spent for added output • **Covering more ground:** devs can do work that previously fell through the cracks of capacity, priority, etc.	• **Drastic increase in context switching,** and fragmented attention • **Increased cognitive load and burnout,** and less capacity for deep work • **Loss of focus:** when everything can be worked on, it's harder to turn down tasks



Research published two weeks ago (5 May) by Microsoft claims AI expands the pool of people who can do high-value work. Our survey found similar, while MS's findings are based on Microsoft 365 Copilot chat usage.



"AI used to do more high-value work" Source: Microsoft

Obviously, Microsoft has an interest in showing the benefits of its AI tools, but in this specific area, it's telling that both studies agree AI does allow devs and non-devs to expand the type of work they do.

Software engineering tradeoffs

Area	✅ Positive impact	❌ Negative impact
Boundaries blur between roles: work across roles increases	• **Devs work at higher abstraction levels**: architecture, product thinking, strategy common – even at entry-level • **Get involved in non-dev work:** it's possible to explore other kinds of work and tasks in the business, design, product, marketing sides • **Managers doing IC work:** non-engineers and hands-off leaders are starting to do hands-on work	• **Loss of the joy of crafting code by hand**, as we do less "manual" coding • **Grief**. Particularly for engineers who value the craft • **Career anxiety:** e.g., personal relevance, the future of the profession, etc. It's a massive change for code to be generated for us • **Being unexcited by "new" work.** Tasks that are unexciting and unfulfilling
Easier to work on unfamiliar stuff like unknown languages, frameworks, tech stacks, etc.	• **Work in unfamiliar environments:** you get more confident about working outside of your expertise without long ramp-up times • **More breadth:** devs can tackle bigger, broader problems than before • **Less reliant on specialists:** reduces dependence on individuals for specialist knowledge	• **Skill atrophy:** forgetting how to do things • **Shallow understanding:** this replaces deep expertise over time • **Foundational skills might not get acquired:** junior engineers skip building up foundational skills that the seniors of today possess

With AI tools, their 'mileage may vary' among individuals. For example, some respondents told us they can spend much more time in a "flow state" thanks to AI tools, as they don't have to wait for input from peers, can keep unblocking themselves, and have fewer interruptions.

But others say the opposite: that because they can start so many more tasks in parallel with AI tools, they keep context switching which knocks them out of a flow state!

The positives and negatives of these tools seem dependent on the environment respondents work in, individuals' personality traits, and where users are on the AI learning curve.

2. Adopting AI at scale is hard

One pattern in the responses is that adopting AI at company-scale remains challenging, including:

- **Costs**: a growing concern covered in depth in Part 1
- **Usage**: getting people to use AI tools continuously is not always straightforward
- **Onboarding and education**: at larger companies, there needs to be support to help devs make the most of the tools
- **Reviewing AI-generated output:** code review is a particular pain point

- **Integrating with internal systems:** AI tools are more helpful when they are seamlessly integrated with internal systems, hence why many larger companies use in-house, deeply embedded coding agents

Adopting AI at company-level doesn't lead to a magical fix for engineering problems, while the benefits of AI in an organizational sense seem to depend on what was in place *before*.

Case in point, AI seems to amplify pre-existing engineering culture. AI doesn't change the underlying *quality* of an organization's engineering culture. Teams with strong engineering practices get more positive benefits out of AI than those without. Teams that see benefits from AI tools already had:

- **Guardrails**: testing and automation around the codebase and deployments
- **Documentation**: they recorded their architectural decisions and engineering practices
- **A quality codebase:** AI agents will replicate patterns already in a codebase

A few quotes on this from respondents to our survey:

> "AI is an amplifier, not a fixer. Good software engineering practices get multiplied. So do the bad ones. Embedding this properly in teams is exciting and important". – *Staff+ engineer at a large company in Europe*

> "I feel like AI allows both faster prototyping and increased velocity on iterations to production software; it relies on existing best practices / project templates our team already have". – *Solutions Architect at a small company in the US*

A workflow that makes one dev "10x" more productive may not work for another. This is another reason why rolling out AI tools doesn't seem to magically make everyone more productive. A senior engineer working at a large company in Canada told us:

> *"It feels like AI workflows are very idiosyncratic in that some people derive (I hate this framing, but...) 10x more productivity benefit from them than other apparently equally clever, educated, and diligent developers. It feels like finding a workflow that clicks with your own habits and heuristics is more important than finding a global optimum for everyone".*

AI amplifies individuals differently, so the team impact is messy to figure out. A US-based principal engineer at a large company reflected on feeling disconnected from colleagues because of how they use AI differently:

> *"I use AI in what I think is probably a more sophisticated way than most of my colleagues, so there can be a disconnect between my work and theirs, which is not good news because I am "The Principal" on the team".*

"The tool that works for you" approach can lead to tooling chaos, even at a team level. While it's empowering to allow devs to choose the AI tools they feel are the most helpful, over time it becomes chaotic when teams can choose their own tech stacks, and when at large companies there are dozens of different technologies. A staff+ engineer at a 200-person business in the Middle East wrote in their response:

> *"We're still trying to figure out how to deal with tooling consistency on a team level. It's one of our biggest struggles, but possibly more due to company structure than anything else. Everyone is using different tools with little coherence. It's been rough."*

Some companies have briefly rolled back AI to deal with the negative effects first. From an engineering lead at a 10,000+ person company in Europe:

> *"Since the AI boom, the quality of technical writing and reasoning from senior engineers in my org has significantly deteriorated. There's an overwhelming volume of low-quality work product that is generated entirely or in part by AI, which has made it very difficult to conduct meaningful review of RFCs or code. We've also seen costly production incidents caused by code written and/or approved by AI, and – while my employer initially bought heavily into the hype – we have now rolled back some of our AI tools to deal with the drop in quality."*

3. Impact on codebase quality

A concrete pattern in our survey data is that codebase quality is decreasing due to AI. The contributing factors are not surprising:

- "AI slop": more low-quality code generated, such as duplicated, verbose code, and poor abstractions
- Too many code reviews, which means review quality slips
- More bugs: due to faster code output and less strict reviews, more bugs sneak into codebases

We discuss the degradation of products and codebases in a recent deepdive, Are AI agents actually slowing us down?

One CTO at a European startup lists their negatives about increased AI usage:

1. *"A lot of tiny bugs and low code quality if you are not careful, verify carefully, and have good structure and guardrails*
2. *AI agents generate too much and repetitive code, making systems harder to maintain*
3. *Developers lose understanding of the codebase and become numb to bad architecture and bad developer experience"*

According to our survey, management often seems unfazed by decreasing quality, and instead focuses on the higher output. A principal DevOps engineer at a

large European company said:

> *"In our company, we hand AI tools to inexperienced engineers who can't distinguish good code from bad code and it's falling on deaf ears in our leadership. They only seem to care about short to mid-term cost savings."*

The maintenance burden of AI-generated code is falling on the fewer engineers who understand and care. A staff engineer at a European company listed the problems that AI-generated code is causing:

- *"Drive by" contributions are up:* many more occasional non-core-engineer contributors adding code but not sharing the maintenance burden
- *Contributing without adding guardrails:* many engineers and most of engineering leadership are not using reasonable guardrails like tests
- *AI slop from folks who have nothing to do with the codebase:* huge volume of slop incoming from people who don't understand the codebase, but will commit and create PRs without fully understanding what they're doing
- *Complexity is exploding:* thanks to the above

The maintenance budget is falling upon fewer devs, while the task of refactoring bloated codebases and reducing complexity is left to those still sufficiently in touch with the codebase, thereby making the maintenance burden even worse.

But some leaders "get it." A CEO at a 20-person company told us:

> *"While AI has made generating code 'cheaper', the monitoring and maintenance worry me; the things that have traditionally cost the most in software. We're increasing the rate of shipping large amounts of code with less understanding and increasing the unpredictability, so how do we work the predictability back on top?"*

There is industry pressure on companies to adopt AI tooling and impose its usage upon engineers, driven by a mix of factors:

- Seeing actual benefits of AI and hearing that other teams and companies enjoy success with it
- Fear of being left behind by competitors, or becoming less relevant
- Anxiety about investor interest if a company is seen as not adopting the latest AI tools.

This often leads to:

- Top-down mandates to use AI
- Expectation of headcount reduction, with smaller teams doing the same amount or more
- Management treating AI productivity gains as a *baseline* rather than a *bonus*.

One staff engineer at a 10,000-person US company explained it like this:

> *"AI is part of almost every work conversation. The entire company expects it to increase productivity and reduce the need to hire people. I keep trying to get better at using it and trying to make it more reliable so I can do more. I do worry about the quality of the work and atrophy of certain skills. It's unclear to me if those skills even matter anymore."*

Pushing AI adoption *blindly* triggers red flags. Respondents shared what makes them worry about things going south in their workplaces:

- *Focus on tracking AI usage, but not the quality of the output.* This will likely lead to product regressions and unhappy customers
- *Pushing for universal adoption.* Some companies target 50%, 80%, or 100% AI usage for certain tasks, seemingly blind to how some targets can worsen the quality of output, or simply create wasteful usage. *See the trend of tokenmaxxing.*
- *A focus on velocity, but without recognition of quality work.* Expecting more velocity and output seems to be the baseline, and there is no recognition for work truly well done.

The "move fast and break things" mantra famously championed by Facebook seems widespread across the industry with AI tools. A senior manager working at a large, European-headquartered company told us:

> *"I see a trend: move fast and break things, and end up breaking things too often. We have to learn to focus on testing and resiliency a lot more, as with AI-driven development we introduce more bugs than before. But the velocity gain is bigger for now".*

Output over quality is leading to the death of code review at some places. As a lead engineer working at a small company summarized:

> *"We're at the death of code review. I used to do very deep code reviews where I'd take the time to understand the architecture and organization and provide feedback on maintainability and efficiency. I have no motivation in spending that time to review a giant PR where it's clear that even the original author didn't bother to do that".*

4. Less experienced engineers and AI

The first generation of software engineers who have never developed without the help of AI are now entering the industry. Here is a response from a young engineer working at a startup as an intern:

> *"I have never worked as a developer without AI. Writing this scares me a bit, actually, but it's the truth!"*

But this will be the new reality for those joining the industry. So, what needs to happen to help a new generation of "AI-native" grad engineers grow professionally?

AI is an amplifier which could amplify the lack of experience. A staff engineer in the US at a large company told us:

> *"Agentic AI is a fascinating mirror. It can code as well as the user who drives it. If that user is a junior engineer, now you have a faster junior engineer. If the user is a staff engineer, now you have a faster staff engineer.*
>
> *What agentic AI doesn't do is magically convert a junior engineer into a staff engineer, because the user driving it still needs enough experience to know what a good solution looks like".*

A junior engineer in Australia shares their experience of how frustrating working with AI tools is:

> *"I think AI agents are great for vibe coding or prototypes where the code quality and functionality doesn't matter that much. I think it's also useful for senior engineers who know what they're doing.*
>
> *For junior engineers like myself, these AI tools are stressful to use. I don't have the experience or knowledge to tell AI exactly what to do or quickly confirm its output, so I spend a lot of time on just triple checking and redoing stuff. I'm overall frustrated, but I'm trying to embrace it as we've been asked to by the company".*

Less experienced engineers seem to use more AI tokens and rack up higher bills. Several respondents observed this: director-level folks noted that junior engineers are in the top-spender category in their orgs, and it is junior devs who spend tokens on unproductive use cases.

There should be more space for junior folks to grow because they use AI more, one staff engineer respondent said:

> *"Companies need to give some breathing room to Junior engineers and help them learn and acquire knowledge using AI tools as a booster and not as a replacement".*

Junior folks seem to be delegated fewer opportunities that could help them grow. This is because senior people can turn to AI for tasks, including those which they would have previously delegated to an intern or new grad. A few responses mention this:

> "AI allows me to have work done that I would usually delegate to a junior or pay a SaaS for; e.g., writing drafts, summarizing the news. " *DevSecOps lead at a small company, Europe*
>
> "I've begun to automate any repetitive task that we previously relied on juniors and offshore contractors for." - *Engineering manager, at a large company, US*

> "I no longer have to delegate work by writing a very long document and briefing a junior engineer." - *Principal engineer, large company, Europe*

Why not consider mentoring junior devs in your organization? It's clear that less experienced engineers are having a rocky start to their careers, so delegating stuff to them instead of to AI could be of high value for newer generations of talent.

5. AI tooling 'addiction'

It seems that the rapid feedback loops of AI-assisted development create addictive tendencies, and there's a noteworthy presence of "addiction lingo" in some responses to our survey:

This post is for paid subscribers

Subscribe

Already a paid subscriber? Sign in



A guest post by

Elin Nilsson

Subscribe to Elin

Hej, I'm Elin! 👋 I do research for The Pragmatic Engineer, various naturey and crafty activities, and think random thoughts about life, the universe and everything ✨

© 2026 Gergely Orosz · Privacy · Terms · Collection notice
Substack is the home for great culture